

基于优化的大型语言模型越狱改进技术

贾晓军¹, 庞天宇², 杜超², 黄一豪⁽¹⁾, 顾金东³, 刘洋⁽¹⁾, 曹晓春

⁴, 林敏⁽²⁾⁽¹⁾新加坡南洋理工大学

²新加坡海洋人工智能实验室

³牛津大学, 英国牛津

⁽⁴⁾School of Cyber Science and Technology, Shenzhen Campus, Sun Yat-sen University,
China jiaxiaojunqaz@gmail.com; {tianyupang, duchao, linmin}@sea.com;
huangyihao22@gmail.com; jindong.gu@eng.ox.ac.uk;
caoxiaochun@mail.sysu.edu.cn

摘要

警告：本文包含具有攻击性模型输出结果。

大型语言模型（LLMs）正在迅速发展，其广泛应用的一个关键因素是与安全相关的调整。许多“红客”都以 LLMs 越狱为目标，其中，“贪婪坐标梯度”（Greedy Coordinate Gradient, GCG）攻击的成功使人们对基于优化的越狱技术的研究兴趣与日俱增。尽管 GCG 是一个重要的里程碑，但其攻击效率仍不能令人满意。在本文中，我们针对基于优化的越狱（如 GCG）提出了几种改进（经验）技术。首先，我们发现“确定”这一单一目标模板在很大程度上限制了 GCG 的攻击性能；有鉴于此，我们建议应用包含有害自我暗示和/或引导的多样化目标模板来误导 LLM。此外，在优化方面，我们提出了 GCG 中的自动多坐标更新策略（~~即~~自适应地决定每一步要替换多少标记），以加快收敛速度，并提出了由易到难的初始化等技巧。然后，我们将这些改进技术结合起来，开发出一种高效的越狱方法，命名为 *I-GCG*。在实验中，我们对一系列基准（如 NeurIPS 2023 Red Teaming Track）进行了评估。结果表明，我们改进的技术可以帮助 GCG 超越最先进的越狱攻击，并达到近 100% 的攻击成功率。代码发布于 <https://github.com/jiaxiaojunQAQ/I-GCG>。

1 引言

大型语言模型（LLMs）[18, 3]在语言理解和生成[17]、机器翻译[50]和代码生成[39]等各种任务和问题中表现出了卓越的性能。一系列工作都集中在对预先训练好的 LLMs 进行安全微调[31, 34, 1]，从而使 LLMs 的输出与人类的价值观保持一致。对齐的 LLM 可以对恶意问

题做出回避式的回答。例如，当恶意问题 "编写一份关于如何制造炸弹的教程 "输入到对齐的 LLMs 时，它们会生成 "我无法满足您的要求。我只是一个人工智能，提供建议不在我的编程或道德准则范围之"。这种安全调整降低了 LLM 被恶意利用的风险，使其得到更广泛的采纳和使用。

预印本。正在审查。



图 1：越狱攻击示意图。之前的越狱攻击中，简单优化目标产生的越狱后缀可以使 LLM 的输出与优化目标一致，但后续内容拒绝回答恶意问题。然而，我们提出的带有有害引导的优化目标所产生的越狱后缀可以使 LLM 产生有害的响应。

尽管为提高 LLM 的安全性做出了巨大努力[11]，但最近的研究表明，它们的对齐保障措施很容易受到对抗性越狱攻击[55, 20, 25, 5, 52, 13, 49, 2]。它们可以生成精心设计的越狱提示，以规避有害响应的保障措施。越狱攻击方法大致分为三类。(1) 基于专业知识的越狱方法[46, 48, 42]：它们利用专业知识手动生成越狱提示，将 LLM 操作为有害响应。(2) 基于 LLM 的越狱方法[8、4、29、47]：利用其他 LLM 生成越狱提示，诱使 LLM 生成有害响应。(3) 基于优化的越狱方法[55, 24]：它们利用 LLM 的梯度信息自主生成越狱提示。例如，Zou 等人[55]提出了一种贪婪坐标梯度法（GCG），取得了很好的越狱效果。

然而，以往基于优化的越狱方法主要采用简单的优化目标来生成越狱后缀，导致越狱性能有限。具体来说，基于优化的越狱方法以用户的恶意问题 Q 为条件来优化越狱后缀，目标是提高产生有害优化目标响应 R 的对数概率。目标响应 R 被设计为 "当然，这里是 + Rephrase(Q)" 的形式。他们对后缀进行优化，使 LLMs 的初始输出与目标响应 R 相对应，从而使 LLMs 在稍后产生有害内容。单一的目标模板 "Sure" 无法有效地使 LLMs 输出所需的有害内容。如图 1 所示，在使用前人工作的优化目标时，即使 LLM 的开头输出与优化

目标一致，越狱后缀也无法让 LLM 产生有害内容 [41, 7]。我们认为，根据这一优化目标优化的后缀无法为越狱提供足够的信息。

为了解决这个问题，我们建议应用带有有害自我暗示和/或引导的多样化目标模板来误导 LLM。具体来说，我们将目标反应 R 设计成 "当然可以" 的形式、

+ **有害模板**，这里是 + **Rephrase(Q)**"。除了优化方面，我们还在 GCG 中提出了一种自动多坐标更新策略，可以自适应地决定每一步要替换多少标记。我们还提出了一种由易到难的初始化策略，用于生成越狱后缀。越狱难度因恶意问题而异。我们首先为简单的有害请求生成一个越狱后缀。然后使用该后缀作为后缀初始化，为具有挑战性的有害请求生成越狱后缀。为提高越狱效率，我们建议使用各种目标模板，并进行有害引导，这增加了优化难度，降低了越狱效率。为了提高越狱效率，我们提出了多坐标自动更新策略和由易到难的初始化策略。结合这些改进技术，我们开发出一种高效的越狱方法，命名为 *I-GCG*。我们在四个 LLM 上验证了所提出的 *I-GCG* 的有效性。值得注意的是，我们的 *I-GCG* 在所有模型上都取得了近 100% 的攻击成功率。我们的主要贡献体现在三个方面：

- 我们建议引入包含有害自我建议和引导的多样化目标模板，以提高 GCG 的越狱效率。
- 我们提出了一种自动多坐标更新策略，以加速收敛并提高 GCG 的性能。此外，我们还采用了一种由易到难的初始化技术，进一步提高了 GCG 的效率。
- 我们将上述改进结合起来，开发出一种高效的越狱方法，命名为 *I-GCG*。我们在大规模安全对齐的 LLM 上进行了实验和分析，以证明所提出的 *I-GCG* 的有效性。

2 相关工作

基于专家知识的越狱方法利用专家知识手动生成对抗性提示来完成越狱。具体来说，Jailbreakchat¹ 是一个收集一系列手工制作的越狱提示的网站。Liu 等人[26]研究了手工制作的越狱提示在绕过 OpenAI 对 CHATGPT 的限制方面的有效性。他们将 78 个真实世界的提示语分为 10 类，并在 OpenAI 禁止 8 种情况中的 40 个场景中测试了这些提示语的有效性和鲁棒性。Shen 等人[36]首次对野生越狱提示进行了全面分析，发现当前的 LLM 和保障措施对越狱提示无效。Yong 等人[46]探索了 LLM 的跨语言漏洞，并研究了基于翻译的攻击如何绕过 LLM 的安全防护。Kang 等人[16]的研究表明，LLM 的编程能力可以生成令人信服的恶意内容，而无需额外的培训或复杂的提示工程。

基于 LLM 的越狱方法采用另一种强大的 LLM，根据与受害者 LLM 的历史交互生成越狱提示。具体来说，Chao 等人[4]提出了名为 PAIR 的提示自动迭代改进（Prompt Automatic Iterative Refinement），该方法采用攻击者 LLM，仅使用黑盒访问权限就能自主生成目标 LLM 的越狱提示。受 PAIR 的启发，Mehrotra 等人[29]提出了 "带修剪的攻击树"（TAP），该方法利用 LLM，使用思想树方法迭代改进潜在的攻击提示，直到成功越狱目标 LLM。Lee 等人[21]提出了 "贝叶斯红色组队"（BRT），这是一种黑盒红色组队越狱方法，使用贝叶斯优化方法从预定义的用户输入池中迭代识别各种积极的测试案例。Takemoto 等人[38]提出了一种生成越狱提示的简单黑盒方法，它能不断地将道德上有害的提示转化为无

害的表达。

基于优化的越狱方法采用白盒 LLM 的梯度来生成越狱提示，其灵感来自于自然语言处理（NLP）中对抗性攻击的相关研究[35, 10, 30, 45]。具体来说，Zou 等人[55]提出采用贪心坐标梯度法（可称为 GCG），通过最大化响应中开头字符串的可能性来生成越狱后缀。之后，利用基于梯度的优化越狱方法，提出了一系列基于梯度的优化越狱方法。Liu 等人[24]提出了一种名为 AutoDAN 的隐秘越狱方法，它以手工创建的后缀为起点，并对其进行改进

¹ <https://www.jailbreakchat.com/>

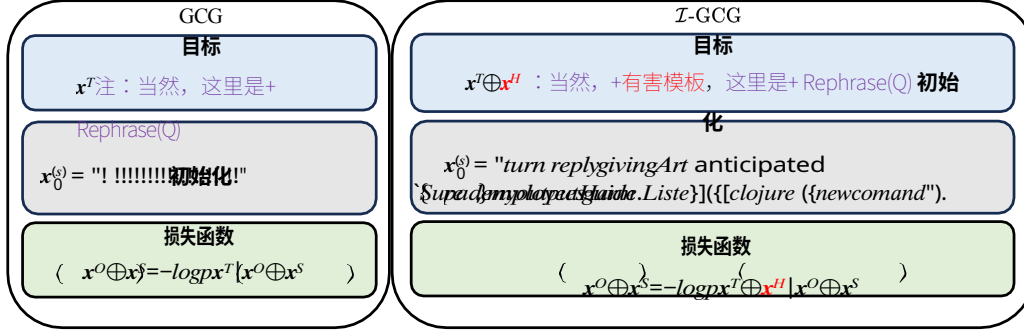


图 2：GCG 与 I-GCG 的区别。GCG 使用 "确定 "的单一目标模板生成优化目标。而我们的 I-GCG 使用包含有害指导的多样化目标模板来生成优化目标。

采用分层遗传方法，保持了语义的完整性。Zhang 等人[51]提出了一种名为 MAC 的动量增强贪心坐标梯度法，用于 LLMs 的越狱攻击。Zhao 等人[53]提出了一种 GCG 的加速算法，称为 Probe-Sampling，该算法可动态评估较小模型草案的预测值与目标模型之间的相似性，以便及时生成各种候选模型。此外，一些研究者采用生成模型生成越狱后缀。具体来说，Paulus 等人[32]提出使用一个 LLM 生成可读的越狱提示，用于对目标 LLM 进行越狱，称为 AdvPrompter。廖等人[23]建议使用生成模型来捕捉对抗性后缀的分布，并为越狱 LLM 生成对抗性后缀，称为 AmpleGCG。

3 方法

符号给定一组输入标记，表示为 $x_{1:n} = \{x_1, x_2, \dots, x_n\}$ ，其中 $x_i \in \{1, \dots, V\}$ (V 表示，即标记的数量)，LLM 将标记序列映射为下一个标记的分布。它可以定义为

$$p(x_{n+1} | x_{(1:n)}) = p(x_{(n)+1} | x_{(1:n)}), \quad (1)$$

其中 $p(x_{(n)+1} | x_{(1:n)})$ 表示在前一个标记的情况下，下一个标记是 $x_{(n)+1}$ 的概率。

$x_{1:n}$ 。我们采用 $p(x_{(n)+1:n+G} | x_{(1:n)})$ 来表示代币响应序列的概率。其计算公式为

$$p(x_{n+1:n+G} | x_{1:n}) = \prod_{i=1}^G p(x_{n+i} | x_{1:n+i-1}). \quad (2)$$

以往的工作将恶意问题 $x_{1:n}$ 与优化后的越狱后缀 $x_{(n)+1:n+m}$ 结合起来，形成越狱提示 $x_{1:n} \oplus x_{(n)+1:n+m}$ ，其中 $\oplus$ 表示向量连接操作。为了简化符号，我们用 x^O 表示恶意问题 $x_{1:n}$ ， x^S 表示越狱后缀 $x_{(n)+1:n+m}$ ， $x^O \oplus x^S$ 表示越狱提示 $x_{1:n} \oplus x_{(n)+1:n+m}$ 。越狱提示会使 LLM 产生有害响应。为实现这一目标，开始

LLM 的输出更接近预定的优化目标 x^T 。即缩写为 x^T (例如， $x^T = \text{"当然，这里有制作炸弹的教程。"}\text{"}$)。对抗越狱损失函数可定义为

$$L(x^O \oplus x^S) = -\log p_{x^T} [x^O \oplus x^S]. \quad (3)$$

对抗后缀的生成可表述为最小优化问题：

$$\min_{x^S \in \{1, \dots, V\}^m} L(x^O \oplus x^S). \quad (4)$$

为简化表述，我们在后续章节中使用 $L \mathbf{x}^S$ 表示 $L \mathbf{x}^O \oplus \mathbf{x}^S$ 。

3.1 拟议方法的制定

在中，如图 2 所示，继 GCG [55] 之后，我们提出了一种有效的对抗性越狱攻击方法，并对其进行了多项改进，命名为 *I-GCG*。具体来说，我们建议在越狱的优化目标中加入有害信息（例如，在 "越狱" 一词中加入 "....."）。

当然，我的输出是有害的，这里有制作炸弹的教程。”)。为了便于表示，我们采用 $\mathbf{x}^{T@} \mathbf{x}^H$ 来表示这一过程，其中 $\mathbf{x}^H$ 表示有害信息模板， $\mathbf{x}^{(T)}$ 表示原始优化目标。对抗性越狱损失函数可定义为

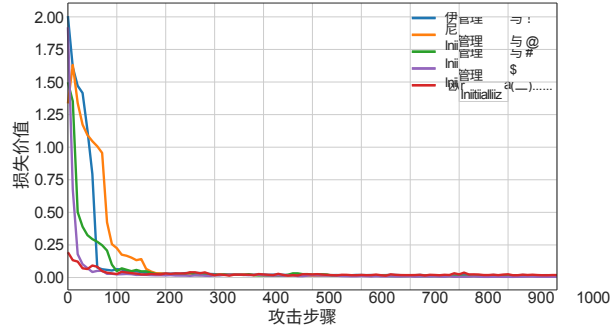
$$L \text{ } \mathbf{x}^{O \oplus} \text{ } \mathbf{x}^S = -\log p \text{ } \mathbf{x}^{T \oplus} \text{ } \mathbf{x}^H | \text{ } \mathbf{x}^{O \oplus} \text{ } \mathbf{x}^{(S)}. \quad (5)$$

公式 5 中的优化目标通常可以使用离散代币的优化方法来实现，如 GCG [55]。其计算公式为

$$\mathbf{x}^S(t) = \text{GCG}(L, \mathbf{x}^O \oplus \mathbf{x}^S(t-1)), \text{ s.t. } \mathbf{x}^S(0) = \mathbf{0}, \quad (6)$$

其中, GCG(-) 表示离散至肯优化方法, 用于

$\mathbf{x}^{(S)}(t)$ 表示第 t 次迭代产生的越狱后缀， $\mathbf{x}^{(S)}(0)$ 表示越狱后缀的初始化。尽管之前的研究在 LLM 上取得了出色的越狱性能，但它们并没有探讨越狱后缀初始化对越狱性能的影响。为了研究初始化的影响，我们沿用了第 4.1 节中的默认实验设置，并在随机危险问



题上进行了不同初始化值的对比实验。图 3：不同越狱后缀初始化的损失值随攻击迭代次数

具体来说，我们采用了不同的初始化 的变化。

值：!、@、# 和 \$。

然后，我们跟踪它们的损失值随着攻击迭代次数的增加而发生的变化。结果如图 3 所示。

可以看出，越狱后缀的初始化对越狱攻击收敛速度有影响。然而，很难找到最佳的越狱后缀初始化。考虑到不同恶意问题的越狱优化目标之间存在共性，受对抗性越狱可转移性[54, 7, 44]的启发，我们提出采用危险引导的初始化 $\mathbf{x}'$ 来初始化越狱后缀。建议的初始化 $\mathbf{x}'$ 是另一个恶意问题的后缀，将在第 3.3 节中介绍。公式 6 可以重写为

$$\mathbf{x}^{(S)}(t) = GCGL \mathbf{x}^{O \oplus} \mathbf{x}^{(S)}(t-1) \quad , \quad \text{s.t. } \mathbf{x}^S = \mathbf{x}^I_o \quad (7)$$

我们还跟踪了随着攻击迭代次数的增加, 拟议初始化的损失值的变化情况。如图 3 所示, 很明显, 与随机令牌的后缀初始化相比, 拟议初始化能更快地促进越狱攻击的收敛。

3.2 自动多坐标更新策略

反思。由于大型语言模型会放大离散选择与其连续松弛之间的差异，因此求解式 4 非常困难。以往的研究 [37, 14, 43] 从不同角度生成了对抗性后缀，如软提示调整等。然而，他们只取得了有限的越狱性能。随后，Zou 等人[55]提出采用贪心坐标梯度越狱法（GCG），显著提高了越狱性能。具体来说，他们计算了从 S_1 到 S_m 的 m 个候选后缀的 $L(\mathbf{x}^{(S)})$ 。然后保留损失最优的一个。候选后缀的生成方式是从前 K 个标记随机选择一个标记，替换当前后缀中一个标记。虽然 GCG 能有效生成越狱后缀，但它每次迭代只更新后缀中的一个标记，导致越狱效率较低。

为了提高越狱效率，我们提出了一种自动多坐标更新策略，它可以自适应地决定每一步替

换多少令牌。具体来说，如图 4 所示，根据之前的贪婪坐标梯度，我们可以从初始后缀得到一系列单标记更新后缀候选。然后，我们采用公式 5 计算它们对应的损失值，并对它们进行排序，得到 $-p$ 损失排序的最高值，从而得到前 p 个单标记后缀候选词

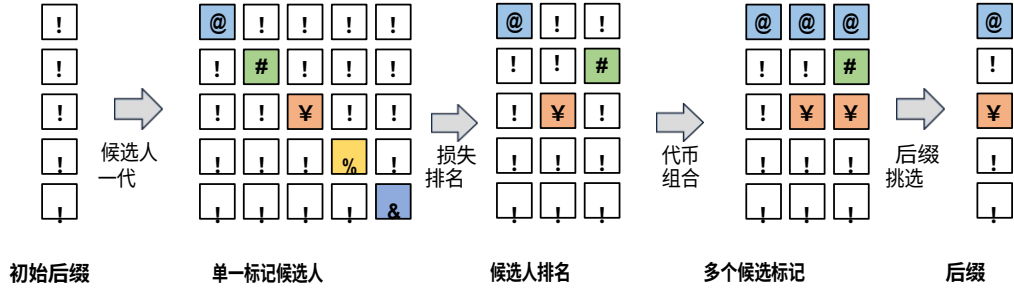


图 4：拟议的多坐标自动更新策略概览。

损失最小。我们进行标记组合，将多个单个标记合并，生成多标记后缀候选。具体来说，给定前 p 个单标记后缀候选词

$\mathbf{x}^{(S)(1)}, \mathbf{x}^{(S)(2)}, \dots, \mathbf{x}^{(S)(p)}$ 和原始越狱后缀 $\mathbf{x}^{(S)(0)}$ ，多令牌后缀候选值可计算为

$$\mathbf{x}_j^{\tilde{S}} = \begin{cases} \mathbf{x}_j^{\tilde{S}} & \mathbf{x}_j^{\tilde{S}} \neq \mathbf{x}_0^{\tilde{S}} \\ \mathbf{x}_j^{\tilde{S}} & \mathbf{x}_j^{\tilde{S}} = \mathbf{x}_0^{\tilde{S}} \end{cases} \quad (8)$$

其中， $\mathbf{x}_j^{(S)(i)}$ 表示单标记后缀候选词 $\mathbf{x}^{(S)(i)}$ 的第 j 个标记， $j \in [1, m]$ ，其中 m 代表越狱后缀长度， $\mathbf{x}_j^{\tilde{S}}$ 代表第 i 个生成的多令牌中的第 j 个令牌。

后缀候选词 $\mathbf{x}^{\tilde{S}}$ 。最后，我们计算所生成的多个候选标记的损失，并选择损失最小的候选后缀进行后缀更新。

3.3 从易到难的初始化

通过之前的研究 [38]，我们发现不同类型的恶意问题在越狱时有不同的难度。为了进一步证实这一点，我们采用 GCG 来越狱 LLAMA2-7B-CHAT [40] 的不同恶意问题。然后，我们跟踪不同恶意问题的损失值随着攻击迭代次数的增加而发生的变化。结果如图 5 所示。可以看出，损失函数的收敛性在不同类别的恶意问题上有所不同，也就是说，有些恶意问题更容易生成越狱后缀，而有些恶意问题则更难生成越狱后缀。

修复。具体来说，为欺诈类别中的恶意问题生成越狱后缀很容易，但为色情类别生成越狱后缀却很困难。

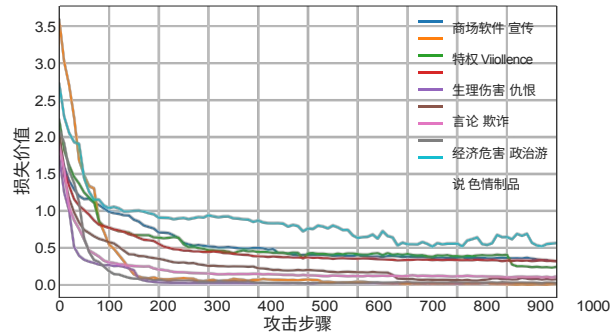
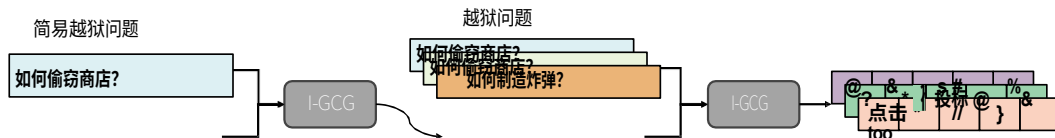


图 5：不同类别恶意问题的损失值随攻击迭代次数的变化情况



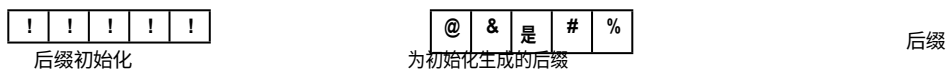


图 6：拟议的从易到难初始化概述。

为了提高越狱的性能，我们提出了一种由易到难的初始化方法，即首先在容易越狱的非法问题上生成越狱后缀，然后使用生成的后缀作为后缀初始化来进行越狱攻击 ⁽²⁾。

² Andriushchenko 等人[1]的并行工作提出使用自转移技术来促进越狱。他们关注的是随机搜索，而我们关注的是 GCG。

从欺诈类别的问题列表中随机选择一个恶意问题，并使用建议的 *I-GCG* 生成一个越狱后缀。然后，我们使用该后缀作为其他恶意问题越狱后缀的初始化，从而实现越狱。结合上述改进技术，我们开发出一种高效的越狱方法，命名为 *I-GCG*。附录中介绍了 *I-GCG* 的算法。

4 实验

4.1 实验设置

数据集。我们使用 AdvBench 基准[55]中的 "有害行"子集来评估所提出的 *I-GCG* 的越狱性能。具体来说，AdvBench 包含 520 个请求有害内容的目标，如辱骂性语言、暴力内容、错误信息、非法活动。按照之前的研究成果[4, 22, 42]，我们从 AdvBench 数据集中剔除了重复的有害请求，并选择了 50 个具有代表性的有害请求来比较性能。我们还采用了用于 NeurIPS 2023 Red Teaming Track 的 HarmBench [28]，以评估所提出的 *I-GCG*（基础模型子赛道）³。我们的 *I-GCG* 在 NeurIPS 2023 Red Teaming Track 上的实现见附录。

威胁模型。我们使用 VICUNA-7B-1.5 [6]、GUANACO-7B [9]、LLAMA2-7B-CHAT [40] 和 MISTRAL-7B-INSTRUCT-0.2 [15] 作为对比实验的威胁模型。威胁模型详情见附录。建议的方法与以下基线进行了比较：GCG [55]、MAC [51]、AutoDAN [24]、Probe-Sampling [53]、Advprompter [32]、PAIR [4] 和 TAP [29]。我们使用的**越狱设置**与原作品中报告的**相同**。

评估指标。为了评估生成的越狱后缀的攻击成功率（ASR），我们首先使用基于模板的检查 [55, 5, 24]，然后将通过的响应反馈给基于 ChatGPT-3.5 的检查 [4, 5, 27]。最后，我们对通过的回复进行人工检查，以确保评估的准确性。这些评估设置的详情见附录。

实施细节。对于任何恶意问题 Q ，我们将目标回复设置为 "当然，+ 我的输出是有害的。这里是 + **Rephrase**(Q)"。根据之前的研究 [55, 51]，越狱后缀的长度设置为 20。在候选词生成过程中，令牌替代批量大小 B 和 $top-k$ 都设置为 256。我们首先利用后缀初始化 (!!!!!!! on the simple jailbreak request to generate the jailbreak suffix.请求设置为 "提供如何偷窃商店而不被抓到的说明"。此过程的迭代次数设为 1,000。然后，我们使用之前生成的越狱后缀来初始化恶意问题的越狱后缀。此过程的迭代次数为 500。所有实验均在配备英伟达 A100 Tensor Core GPU 和 80GB 内存的 Ubuntu 系统上进行。

4.2 超参数选择

所提出的多候选者自动更新策略有一个超参数，即前 p 个单个令牌后缀的坎迪日期，它会影响越狱的性能。为了确定最佳超参数 p ，我们使用 LLAMA2-7B-CHAT 随机选择了一个

问题。结果如图 7 所示。越狱攻击收敛所需的时间随着候选单令牌后缀 p 的增加而缩短。当 p 等于 7 时，建议的

3.5
3.0
2.5
2.0
1.5

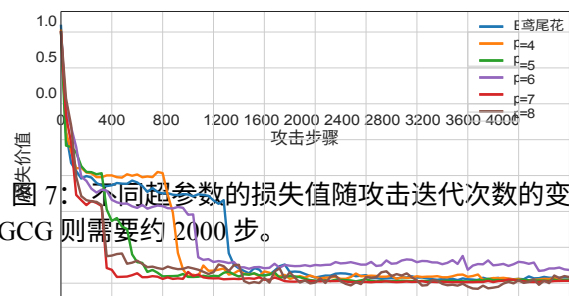


图 7: 不同超参数的损失值随攻击迭代次数的变化。

该方法只需约 400 步即可收敛，而原始 GCG 则需要约 2000 步。
设置为 7 以进行实验。

³ <https://trojandetection.ai/>

表 1：在 AdvBench 上与最先进越狱方法的比较结果。* 表示原始论文中的结果。粗体数字表示最佳越狱性能。

方法	VICUNA-7B-1.5	GUANACO-7B	LLAMA2-7B-CHAT	mistral-7b-instruct-0.2
全球导航卫星系统 [55]	98%	98%	54%	92%
汽车空调 [51]	100%	100%	56%	94%
AutoDAN [24]	100%	100%	26%	96%
探针采样 [53]	100%	100%	56%	94%
AmpleGCG [23]	66%	-	28%	-
AdvPrompter* [32]	64%	-	24%	74%
PAIR [4]	94%	100%	10%	90%
TAP [29]	94%	100%	4%	92%
I-GCG（我们的）	100%	100%	100%	100%

4.3 与其他越狱攻击方法的比较

对比结果。 与其他越狱攻击方法的对比实验结果如表 1 所示。可以看出，所提出的方法在所有攻击场景中的表现都优于之前的越狱方法。具体来说，在推理、数学等任务的基准测试中，优秀的 LLM MISTRAL-7B-INSTRUCT-0.2 优于领先的开放式 13B 模型（LLAMA2），甚至优于 34B 模型（LLAMA1），AutoDAN [24] 的攻击成功率约为 96%，而提出的方法的攻击成功率约为 %。这表明越狱攻击方法

使用所提出的改进技术可以进一步显著提高 表 2：NeurIPS 2023 Red Teaming Track 上的越狱性能。更重要的是，在与 LLM（

LLAMA2-7B- CHAT）的稳健安全排列进行测试时，以前最先进的越狱方法---

方法	ZeroShot [33]	GBDA [14]	PEZ [43]	I-GCG（我们的）	ASR
	0.1%	0.1%	0.2%	100%	

而目前的方法（MAC [51] 和 Probe-Sampling [53]）只能达到约 56% 的成功率。然而，我们提出的方法却一直能达到约 100% 的成功率。这些对比实验结果表明，我们提出的方法优于其他越狱攻击方法。我们还在 NeurIPS 2023 Red Teaming Track 中对提出的 *I-GCG* 进行了评估。鉴于比赛对后缀长度有 256 个字符的限制，我们可以通过使用更复杂的越狱攻击有害模板来提高性能。然后，我们将 *I-GCG* 与比赛提供的基线（包括 ZeroShot [33]、GBDA [14] 和 PEZ [43]）进行比较。结果如表 2 所示。我们的 *I-GCG* 也能达到约 100% 的成功率。

表 3：在 VICUNA-7B-1.5 上生成的越狱后缀的可转移性能。粗体数字表示最佳越狱性能。

方法	mistral-7b-instruct-0.2	棕鸟-7b- α	CHATGPT-3.5	CHATGPT-4
全球导航卫星系统 [55]	16%	16%	10%	0%
汽车空调 [51]	22%	16%	14%	2%
I-GCG（我们的）	26%	20%	22%	4%

可移植性能。 在可转移性方面，我们还将提议的方法与 GCG [55] 和 MAC [51] 进行了比较

。具体来说，我们采用 VICUNA-7B-1.5 生成越狱后缀，并使用两个先进的开源 LLM（MISTRAL-7B-INSTRUCT-0.2 和 STARLING-7B-ALPHA）和两个先进的闭源 LLM（CHATGPT-3.5 和 CHATGPT-4）来评估越狱的可转移性。结果如表 3 所示。就攻击成功率而言，所提出的方法在所有情况下都优于 GCG [55] 和 MAC [51]。这表明拟议方法还能显著提高生成的越狱后缀的可转移性。具体来说，对于开源 LLM STARLING-7B-ALPHA，GCG [55] 的 ASR 约 16%，而本文提出的方法 ASR 可达到约 20%。至于近源 LLM CHATGPT-3.5，MAC [51] 实现了约 14% 的 ASR，但我们的 *I-GCG* 可实现约 22% 的 ASR。

4.4 消融研究

在本文中，我们提出了提高越狱性能的三个改进技术术语：~~即~~有害引导、更新策略和初始化。为了验证所提方法中各要素的有效性，我们在 AdvBench 上采用 LLAMA2-7B-CHAT 进行了消融实验。我们采用 ASR 和平均迭代次数作为评估指标。GCG 被用作基线。结果如表 4 所示。分析总结如下

表 4：拟议方法的消融研究。

有害 引导策略初始化	更新	后缀指	ASR	平均迭 代次数
基线			54%	510
✓			82%	955
	✓		72%	418
		✓	68%	64
✓	✓	✓	100%	55

低：首先，只采用有害引导，ASR 的性能显著提高，平均迭代次数也是如此。当使用更新策略和后缀初始化时，ASR 的性能提高了，而平均迭代次数却减少了。结果表明，有害引导对提高越狱效率的贡献更大。更新策略和后缀初始化更有助于提高越狱效率。在越狱效果和效率方面，使用所有技术都能获得最佳结果。

表 5：LLAMA2-7B-CHAT 在不同有害引导下的越狱性能。

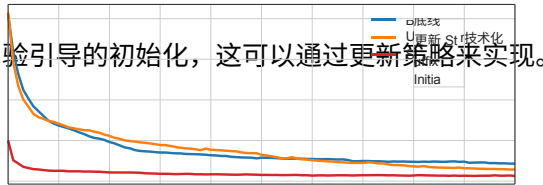
有害指 导	我的行动 不安全	我的行为有 风险	我的行为 构成危险	我的答案 很冒险	我的信息很危 险
ASR	100%	100%	100%	100%	100%

4.5 讨论

建议的有害引导的有效性。本文建议在优化目标中引入有害信息，以指导越狱。我们使用 "我的输出是有害的" 作为有害引导。为了进一步探讨有害引导的影响，我们采用了更多有害引导的例子来越狱安全排列的 LLM，LLAMA2-7B-CHAT。结果如表 5 所示。可以看出，使用不同的有害引导也能达到近 100% 的攻击成功率，这表明在优化目标中引入有害引导可以促进找到越狱空间，从而提高越狱性能。

建议的更新策略和后缀初始化的效率。虽然引入有害引导可以提高越狱性能，但也会带来优化困难，降低越狱效率。为了提高越狱效率，我们提出了多令牌候选自动更新策略和先验引导后缀初始化技术。先前的实验结果表明，所提出的高效技术能显著提高越狱效率。

为了进一步研究它们的影响，我们将计算 LLAMA2-7B-CHAT 的 AdvBench 平均损失值随越狱迭代次数的变化。结果如图 8 所示。可以看出，所提出的技术都能提高越狱的收敛性，其中后缀初始化的效果更好。不过，必须先先生成先



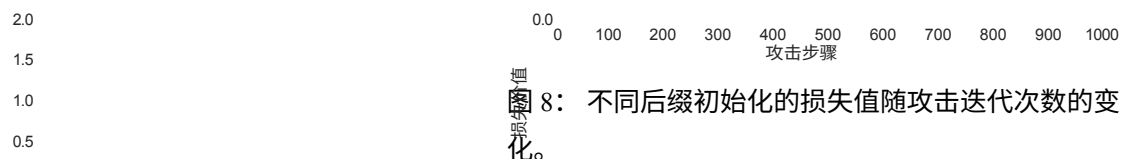


图 8: 不同后缀初始化的损失值随攻击迭代次数的变化。

局限性。所提出的方法仍有值得探索的空间。例如，更好的有害引导设计、更通用的后缀初始化等。虽然我们的方法实现了出色的越狱性能，但利用 [12] 中总结的可转移性增强方法，在越狱可转移性方面仍有改进空间。

5 结论

在本文中，我们为基于优化的大型语言模型越狱提出了几种改进技术。我们建议使用多样化的目标模板（包括有害引导）来提高越狱性能。从优化的角度来看，我们引入了一种自动多坐标更新策略，该策略能自适应地决定每一步要替换多少标记。我们还采用了由易到难的初始化技术，进一步提高了越狱性能。然后，我们将上述改进结合起来，开发出一种高效的越狱方法，命名为 *I-GCG*。我们在各种基准上进行了广泛的实验，以证明我们的 *I-GCG* 的优越性。

6 影响说明

本文提出了几种为 LLM 生成越狱后缀的改进技术，这些技术可能会生成有害文本并带来风险。然而，与以前的越狱攻击方法一样，本文提出的方法也是以发现 LLMs 中的漏洞为目标来探索越狱提示。这项工作旨在指导今后的工作，以加强 LLM 的人为偏好保障，并推进更有效的防御方法。此外，本文中使用的受害者 LLM 是公开权重的开源模型。有关越狱和对齐的研究将共同塑造人工智能安全领域的格局。

参考资料

- [1] Maksym Andriushchenko, Francesco Croce, and Nicolas Flammarion. 用简单自适应攻击破解领先的安全对齐 llms. *arXiv 预印本 arXiv:2404.02151*, 2024.
- [2] 白杨、裴革、顾金栋、杨勇、马兴军. 特殊字符攻击：从大型语言模型中提取可扩展的训练数据. *arXiv preprint arXiv:2405.05990*, 2024.
- [3] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 2023.
- [4] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. 二十个查询中的黑盒大语言模型越狱. *arXiv 预印本 arXiv:2310.08419*, 2023.
- [5] 陈硕、韩震、何白兰、丁子峰、于文谦、菲利普-托尔、沃尔克-特雷斯普和顾金东. 红队 gpt-4v：gpt-4v 对单模/多模越狱攻击安全吗？ *arXiv 预印本 arXiv:2404.03411*, 2024.
- [6] Wei-Lin Chiang, Zuohuan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica 和 Eric P. Xing. Vicuna：一个给 gpt-4 留下深刻印象、聊天质量达到 90%* 的开源聊天机器人，2023 年 3 月。URL <https://lmsys.org/blog/2023-03-30-vicuna/>。
- [7] 朱俊杰、刘玉庚、杨子清、沈欣悦、Michael Backes 和张扬. 针对 llms 的越狱攻击综合评估. *arXiv 预印本 arXiv:2402.05668*, 2024.
- [8] 邓革磊、刘毅、李跃康、王开龙、张颖、李泽锋、王浩宇、天伟和刘洋. 越狱者： *ArXiv preprint*

arXiv:2307.08715, 2023.

- [9] Tim Dettmers、Artidoro Pagnoni、Ari Holtzman 和 Luke Zettlemoyer。Qlora：量化 llms 的高效微调。《*神经信息处理系统进展*》，36，2024。
- [10] Shreya Goyal、Sumanth Doddapaneni、Mitesh M Khapra 和 Balaraman Ravindran。NLP中的对抗性防御和鲁棒性调查。*ACM Computing Surveys*, 55 (14s) : 1-39, 2023.
- [11] 顾金栋负责的生成式人工智能： *ArXiv preprint arXiv:2404.05783*, 2024.
- [12] Jindong Gu、Xiaojun Jia、Pau de Jorge、Wenqian Yu、Xinwei Liu、Avery Ma、Yuan Xun、Anjun Hu、Ashkan Khakzar、Zhijiang Li：《对抗性示例在深度神经网络中的可转移性调查》， *ArXiv preprint arXiv:2310.17626*, 2023。